Python — Interview Questions

Python questions asked in fresher technical interviews — data types, functions, OOP, exceptions and the “how does it work” questions — with short answers and small code examples (all tested on Python 3).

Answer like this in the interview

Define in one line → **explain** with one real-world example → add **one detail** (a difference, a use case or a limitation). Stop at 30–45 seconds and let the interviewer ask the follow-up.

Questions marked ★ **MOST ASKED** come up in almost every fresher technical round — learn those first.

Top questions to learn first

Features of Python · List vs tuple vs set vs dict · Mutable vs immutable · is vs == · *args and **kwargs · Lambda, map, filter · Decorators · Generators & yield · Shallow vs deep copy · Exception handling

1 · Basics

1 What is Python? What are its key features? ★ MOST ASKED

A **high-level, interpreted, general-purpose** language created by Guido van Rossum (1991).

Features: simple readable syntax, **dynamically typed**, **interpreted** (runs line by line through the interpreter), object-oriented and functional, huge standard library, cross-platform, automatic memory management. Used in web development (Django, Flask), data science (NumPy, Pandas), ML (scikit-learn, TensorFlow), automation and scripting.

2 Is Python compiled or interpreted?

Both, in a sense: the source `.py` is first **compiled to bytecode** (`.pyc`), which is then executed by the **Python Virtual Machine** (interpreter). We call it interpreted because there is no separate compile step for the programmer.

3 What does dynamically typed mean?

Types are checked at **run time** and a variable can hold any type — `x = 10` then `x = "hi"` is valid. You don't declare types (type hints like `x: int` are optional and not enforced). Python is also **strongly typed**: `"5" + 5` raises a TypeError instead of converting silently.

4 What is PEP 8? What is indentation used for?

PEP 8 is the official style guide (4-space indentation, snake_case names, lines ≤ 79 characters). Python uses **indentation to define blocks** instead of braces — wrong indentation causes an `IndentationError`.

2 · Data types & data structures

5 What are the built-in data types?

Numeric: `int`, `float`, `complex` · Text: `str` · Sequence: `list`, `tuple`, `range` · Mapping: `dict` · Set: `set`, `frozenset` · Boolean: `bool` · Binary: `bytes`, `bytearray` · `NoneType` (`None`).

6 Mutable vs immutable types? ★ MOST ASKED

Mutable objects can be changed in place: `list`, `dict`, `set`, `bytearray`. **Immutable** objects cannot: `int`, `float`, `str`, `tuple`, `frozenset`, `bool`.

Changing an immutable value creates a **new object**. Only immutable (hashable) objects can be dictionary keys or set elements.

List vs tuple vs set vs dictionary ★ MOST ASKED

Point	List	Tuple	Set	Dictionary
Syntax	[1, 2, 3]	(1, 2, 3)	{1, 2, 3}	{'a': 1}
Ordered	Yes	Yes	No	Yes (insertion order, 3.7+)
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys unique
Indexing	Yes	Yes	No	By key
Use	Changing collection	Fixed data, faster, dict keys	Unique items, membership tests	Key–value lookup

7 append() vs extend(); remove() vs pop() vs del; sort() vs sorted()?

- `append(x)` adds x as **one element**; `extend(iterable)` adds **each element**. `[1].append([2,3])` → `[1, [2, 3]]`; `[1].extend([2,3])` → `[1, 2, 3]`.
- `remove(value)` deletes the first matching value; `pop(index)` deletes and **returns** the item (last by default); `del a[i]` deletes by index or slice.
- `list.sort()` sorts **in place** and returns None; `sorted(iterable)` returns a **new** sorted list and works on any iterable.

8 What is slicing? How do you reverse a string or list?

`seq[start:stop:step]` — stop is excluded. `s[::-1]` reverses, `s[1:4]` gives index 1 to 3, `s[-1]` is the last element. Slicing a list returns a new (shallow) copy.

```
s = "python"
print(s[1:4])    # yth
print(s[::-1])   # nohtyp
print(s[-3:])    # hon
```

9 What is a list comprehension? Dict and set comprehensions? ★ MOST ASKED

A compact way to build a collection from an iterable with an optional condition — usually faster and more readable than a loop.

```
squares = [x * x for x in range(6) if x % 2 == 0]   # [0, 4, 16]
lengths = {w: len(w) for w in ["hi", "code"]}        # {'hi': 2, 'code': 4}
unique  = {c for c in "banana"}                      # {'a', 'b', 'n'}
```

3 · Identity, copying & memory

10 What is the difference between `is` and `==`? ★ MOST ASKED

`==` compares **values**; `is` compares **identity** (same object in memory, same `id()`). Use `is` mainly for `None` checks, e.g. `if x is None`.

```
a = [1, 2]; b = [1, 2]; c = a
print(a == b)   # True  — same values
print(a is b)   # False — different objects
print(a is c)   # True  — same object
```

11 Shallow copy vs deep copy? ★ MOST ASKED

Assignment `b = a` copies only the reference. A **shallow copy** (`copy.copy`, `a[:]`, `list(a)`) creates a new outer object but **shares nested objects**. A **deep copy** (`copy.deepcopy`) copies everything recursively.

```
import copy
a = [[1, 2], [3]]
s = copy.copy(a); d = copy.deepcopy(a)
a[0].append(99)
print(s[0])   # [1, 2, 99] — shared inner list
print(d[0])   # [1, 2]     — independent
```

12 How is memory managed in Python?

Objects live in a **private heap** managed by the interpreter. Memory is freed mainly by **reference counting** (an object is deleted when its count reaches 0), and a **cyclic garbage collector** (`gc` module) cleans up reference cycles. You never call free() yourself.

4 · Functions

13 What are *args and **kwargs? ★ MOST ASKED

`*args` collects extra **positional** arguments into a **tuple**; `**kwargs` collects extra **keyword** arguments into a **dictionary**. They let a function accept any number of arguments.

```
def show(*args, **kwargs):
    print(args, kwargs)

show(1, 2, name="Asha", age=21)   # (1, 2) {'name': 'Asha', 'age': 21}
```

14 What is a lambda? Explain map, filter and reduce. ★ MOST ASKED

A **lambda** is a small anonymous one-expression function. `map` applies a function to every item, `filter` keeps items where the function is True, `reduce` (from functools) combines items into one value.

```
from functools import reduce
nums = [1, 2, 3, 4]
print(list(map(lambda x: x * 2, nums)))          # [2, 4, 6, 8]
print(list(filter(lambda x: x % 2 == 0, nums)))  # [2, 4]
print(reduce(lambda a, b: a + b, nums))          # 10
```

15 Why is a mutable default argument dangerous?

Default values are created **once, when the function is defined**, so a list default is shared across calls. Use `None` and create the list inside.

```
def add(item, bucket=[]):        # bug: same list every call
    bucket.append(item); return bucket
print(add(1), add(2))            # [1, 2] [1, 2]

def add_ok(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item); return bucket
print(add_ok(1), add_ok(2))      # [1] [2]
```

16 How are arguments passed — by value or by reference?

By **object reference** (“pass by assignment”). The function receives a reference to the same object: changing a **mutable** object (appending to a list) is visible outside; reassigning the parameter or “changing” an immutable value is not.

17 What is variable scope (LEGB)? global vs nonlocal?

Names are looked up in order **L**ocal → **E**nclosing function → **G**lobal → **B**uilt-in. `global x` lets a function assign to a module-level variable; `nonlocal x` lets a nested function assign to the enclosing function's variable.

---

5 · Advanced features

18 What is a decorator? ★ MOST ASKED

A function that **takes another function and returns a new function** that adds behaviour (logging, timing, authentication) without changing the original code. Applied with `@decorator`.

```
def log(func):
    def wrapper(*args, **kwargs):
        print("Calling", func.__name__)
        return func(*args, **kwargs)
    return wrapper

@log
def greet(name):
    return "Hello " + name

print(greet("Ravi"))   # Calling greet  /  Hello Ravi
```

---

19 What is a generator? yield vs return? ★ MOST ASKED

A generator is a function that uses `yield` to produce values **one at a time, lazily**, pausing between them — it saves memory for large or infinite sequences. `return` ends the function and sends back one value; `yield` pauses it and resumes from the same point on the next call.

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1

print(list(countdown(3)))   # [3, 2, 1]
gen = (x * x for x in range(3))   # generator expression
print(next(gen), next(gen))       # 0 1
```

---

20 Iterable vs iterator?

An **iterable** can be looped over (list, str, dict) — it has `__iter__()`. An **iterator** produces the next value with `__next__()` and remembers its position; `iter(list)` gives an iterator, and it raises `StopIteration` when exhausted. Every generator is an iterator.

---

21 What is the GIL? Threading vs multiprocessing?

The **Global Interpreter Lock** in CPython lets only **one thread run Python bytecode at a time**. So threads help with **I/O-bound** work (network, files) but not CPU-heavy work. For **CPU-bound** tasks use `multiprocessing` (separate processes, each with its own interpreter).

22 What is `if __name__ == "__main__":` for?

`__name__` is `"__main__"` when the file is run directly, and the module name when it is imported. Code under this check runs only when the file is executed as a script, not when imported.

6 · OOP in Python

23 What are `__init__` and `self`? ★ MOST ASKED

`__init__` is the **initialiser** (constructor-like method) called when an object is created. `self` is the reference to the **current object**; it is passed automatically as the first parameter of instance methods.

```
class Student:
    school = "T.I.M.E."              # class variable — shared
    def __init__(self, name):
        self.name = name             # instance variable — per object
    def __str__(self):
        return f"Student({self.name})"

s = Student("Kavya")
print(s, Student.school)            # Student(Kavya) T.I.M.E.
```

Instance method vs @classmethod vs @staticmethod

Type	First parameter	Can access	Typical use
Instance method	self	Object and class data	Normal behaviour of an object
@classmethod	cls	Class data (not one object)	Alternative constructors, e.g. from_string()
@staticmethod	none	Neither (just a function in the class)	Utility/helper related to the class

24 Does Python support multiple inheritance? What is MRO?

Yes — `class C(A, B)`. The **Method Resolution Order** decides which parent's method is used, using the **C3 linearisation** (left to right, parents after children). View it with `C.__mro__` or `C.mro()`. `super()` follows the MRO.

25 What are dunder (magic) methods? `__str__` vs `__repr__`?

Special methods with double underscores that let objects work with built-in operations: `__init__`, `__len__`, `__add__` (for `+`), `__eq__`, `__getitem__`. `__str__` gives a **readable** string for users (`print`); `__repr__` gives an **unambiguous** developer string, used in the shell and as a fallback.

26 How do you make attributes private in Python?

There is no true private. `_name` signals internal use by convention; `__name` triggers **name mangling** to `_ClassName__name`, which prevents accidental access. Use `@property` to add a getter/setter with validation.

7 · Exceptions, files & modules

27 Explain exception handling: try, except, else, finally, raise. ★ MOST ASKED

`try` holds risky code; `except` handles a specific error; `else` runs **only if no exception** occurred; `finally` **always** runs (cleanup). `raise` throws an exception yourself; create custom exceptions by subclassing `Exception`.

```
def divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        return "Cannot divide by zero"
    else:
        return result
    finally:
        print("done")

print(divide(10, 2))   # done / 5.0
print(divide(1, 0))    # done / Cannot divide by zero
```

28 Name common built-in exceptions.

`ValueError` (right type, wrong value — `int("abc")`), `TypeError` (wrong type — `"5" + 5`), `IndexError`, `KeyError`, `ZeroDivisionError`, `NameError` (undefined variable), `AttributeError`, `FileNotFoundError`, `ImportError`.

29 How do you read and write files? Why use `with`?

`open(path, mode)` with modes `r`, `w` (overwrite), `a` (append), `rb`/`wb` (binary). The `with` **statement** (context manager) closes the file automatically, even if an error occurs.

```
with open("notes.txt", "w") as f:
    f.write("line 1\n")
with open("notes.txt") as f:
    for line in f:
        print(line.strip())
```

30 Module vs package vs library? What are pip and a virtual environment?

A **module** is one `.py` file; a **package** is a folder of modules (with `__init__.py`); a **library** is a collection of packages (NumPy, Requests). **pip** installs packages from PyPI. A **virtual environment** (`python -m venv env`) gives each project its own isolated set of packages and versions.

If your resume says data science / ML

Expect: NumPy array vs list (faster, fixed type, vectorised operations), Pandas Series vs DataFrame, handling missing values (`dropna`, `fillna`), `groupby`, and how you used these in your project.

Rapid-fire one-liners

Question	One-line answer
What is `None`?	A special singleton meaning “no value”; its type is NoneType.
What is `pass`?	A no-op placeholder where a statement is required.
break vs continue?	break exits the loop; continue skips to the next iteration.
What does `range()` return?	A lazy range object, not a list (Python 3).
Can a tuple contain a list?	Yes — the tuple is immutable but the list inside can change.
What is `enumerate()`?	Gives (index, item) pairs while looping.
What is `zip()`?	Pairs items from several iterables: zip([1,2], 'ab') → (1,'a'), (2,'b').
What is pickling?	Converting an object to bytes (pickle.dump) and back (pickle.load).
What is `//` vs `/`?	/ is true division (float); // is floor division.
What does `join()` do?	Joins an iterable of strings with a separator: '-'.join(['a','b']) → 'a-b'.

Common mistakes interviewers catch

- Saying tuples are fully unchangeable when they hold a mutable object — only the tuple's references are fixed.
- Using `is` to compare numbers or strings — use ==.
- Using a list as a default argument.
- Saying Python threads run CPU work in parallel — the GIL prevents it.
- Saying `sort()` returns the sorted list — it returns None.